

|                  |                                                                                    |
|------------------|------------------------------------------------------------------------------------|
| Experiment No.   | 1                                                                                  |
| Experiment Title | Working with Python packages – NumPy, SciPy, Scikit-Learn, Matplotlib <sup>1</sup> |
| Student Name     | Pranav N                                                                           |
| Register Number  | 3122247001044                                                                      |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                                       |
| Submission Date  | Aug 2026                                                                           |

## 1 Objective

The objective of this experiment is to explore the core Python libraries used in machine learning – NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib – and to understand key operations such as array manipulation, data preprocessing, mathematical computing, machine learning workflows, and data visualization. In addition, the experiment aims to explore public dataset repositories such as the UCI Machine Learning Repository and Kaggle, identify the appropriate machine learning task (classification, regression, supervised, unsupervised) for a variety of real-world datasets, and implement an end-to-end machine learning workflow – covering data loading, exploratory data analysis, preprocessing, feature selection, and data splitting – on a chosen dataset.

## 2 Problem Statement

Given a raw, real-world dataset, the task is to build a generic, reusable machine learning pre-processing pipeline that:

- **Input:** A tabular dataset (CSV/Excel) containing numeric and categorical features along with a target column.
- **Process:** Load the dataset, perform exploratory data analysis (EDA), handle missing values and duplicates, encode categorical variables, scale numeric features, automatically detect whether the task is a classification or regression problem, select the most relevant features using statistical feature-selection techniques, and split the data into training, validation, and test sets.
- **Output:** A cleaned, encoded, scaled, and feature-selected dataset split into train/-validation/test partitions, ready to be consumed by a downstream machine learning model.

For this experiment, the pipeline was demonstrated on the **Loan Amount Prediction** dataset, where the goal is to predict the `loan_amount` sanctioned to an applicant based on their demographic and financial attributes.

## 3 Dataset Description

|              |                                                                                           |
|--------------|-------------------------------------------------------------------------------------------|
| Dataset Name | Loan Approval / Loan Amount Prediction Dataset ( <code>loan_approval_dataset.csv</code> ) |
|--------------|-------------------------------------------------------------------------------------------|

|                    |                                                                                          |
|--------------------|------------------------------------------------------------------------------------------|
| Dataset Source     | UCI Machine Learning Repository / Kaggle Datasets                                        |
| Number of Samples  | 4269                                                                                     |
| Number of Features | 13 (including target column and identifier);<br>10 features selected after preprocessing |
| Number of Classes  | Not applicable – regression task (continuous target: <code>loan_amount</code> )          |
| Missing Values     | None found                                                                               |
| Train-Test Split   | Train: 2988 (70%), Validation: 427 (10%),<br>Test: 854 (20%)                             |

## 4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed on the loan dataset to understand the distribution, relationships, and quality of the data before preprocessing. The dataset consists of 4269 rows and 13 columns, with columns such as `loan_id`, `no_of_dependents`, `education`, `self_employed`, `income_annum`, `loan_amount`, `loan_term`, `cibil_score`, `residential_assets_value`, `commercial_assets_value`, `luxury_assets_value`, `bank_asset_value`, and `loan_status`. No missing values and no duplicate rows were found in the dataset.

- **Dataset overview:** 4269 rows  $\times$  13 columns; a mix of numeric (income, assets, credit score, loan amount/term) and categorical (education, self-employment status, loan status) columns.
- **Statistical summary:** Mean `income_annum`  $\approx$  \$5.06 million, mean `loan_amount`  $\approx$  \$15.13 million, mean `cibil_score`  $\approx$  600, with wide standard deviations indicating high variability in applicant profiles.
- **Missing value analysis:** No missing values were detected in any column.
- **Class distribution:** The categorical columns `education`, `self_employed`, and `loan_status` were visualized; `loan_status` is imbalanced with more "Approved" ( 2656) than "Rejected" ( 1613) applications.
- **Correlation matrix:** Strong positive correlations exist between `income_annum`, `loan_amount`, and asset-value columns (e.g., `luxury_assets_value`), while `cibil_score` shows negligible correlation with other numeric features.
- **Feature distribution:** Histograms and box plots reveal that asset-value columns (residential, commercial, luxury, bank) are right-skewed with several high-value outliers, while `income_annum`, `loan_term`, and `cibil_score` are approximately uniformly distributed.

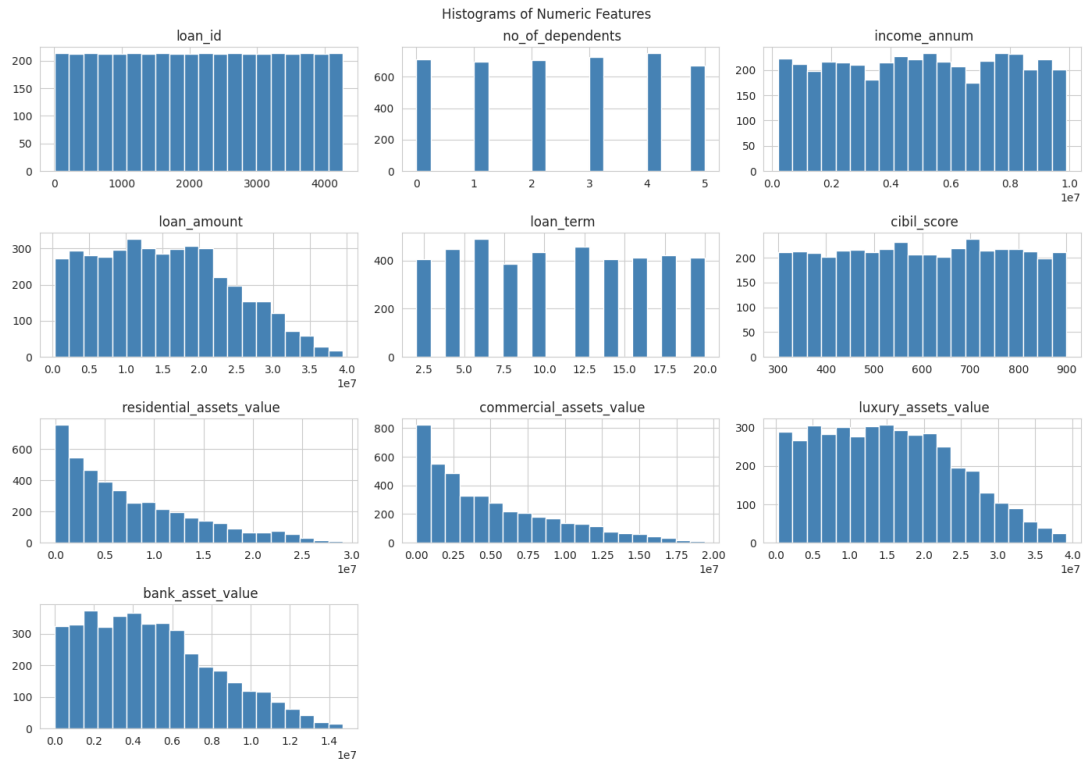


Figure 1: Histograms of all numeric features in the loan dataset.

**Inference:** The histograms show that `loan_id` and `cibil_score` are approximately uniformly distributed, indicating no bias in ID assignment or credit score sampling. `loan_amount` and `income_annum` are right-skewed with a long tail toward higher values, reflecting a small number of high-income, high-loan applicants. The asset-value columns (residential, commercial, luxury, bank assets) are heavily right-skewed, with the majority of applicants holding relatively low asset values and a few holding very high values. This skew motivates the standardization step performed later in preprocessing.

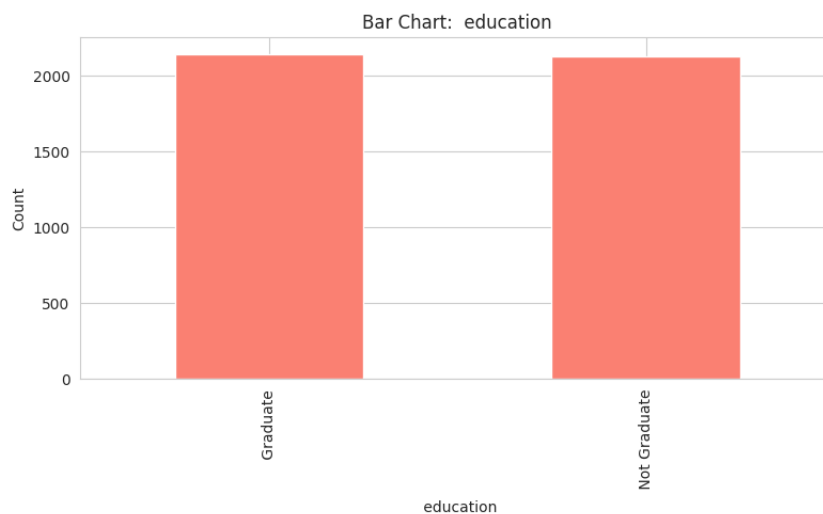


Figure 2: Bar chart of applicant education level (Graduate vs. Not Graduate).

**Inference:** The applicant pool is almost evenly split between "Graduate" and "Not

Graduate” applicants (roughly 2150 vs. 2119), indicating that education level alone is unlikely to introduce major class imbalance in the dataset. This near-balanced distribution makes **education** a reasonable categorical feature for one-hot encoding without needing special handling for rare categories.



Figure 3: Bar chart of applicant self-employment status (Yes vs. No).

**Inference:** Similar to education, the self-employment status is nearly evenly split between “Yes” and “No” applicants. This balance suggests that self-employment status can contribute meaningful, unbiased signal to the model without one category dominating the training data.

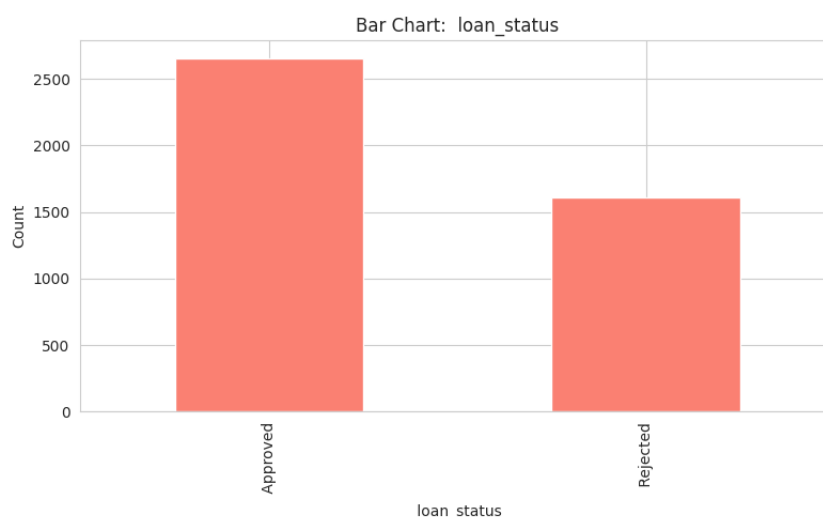


Figure 4: Bar chart of loan status (Approved vs. Rejected).

**Inference:** The **loan\_status** column is moderately imbalanced, with approximately 62% of loans “Approved” and 38% “Rejected”. Although **loan\_status** is not the prediction target in this regression setup (it is instead used as an input feature after one-hot encoding), this imbalance is worth noting if the same dataset were later used for a classification task predicting loan approval.

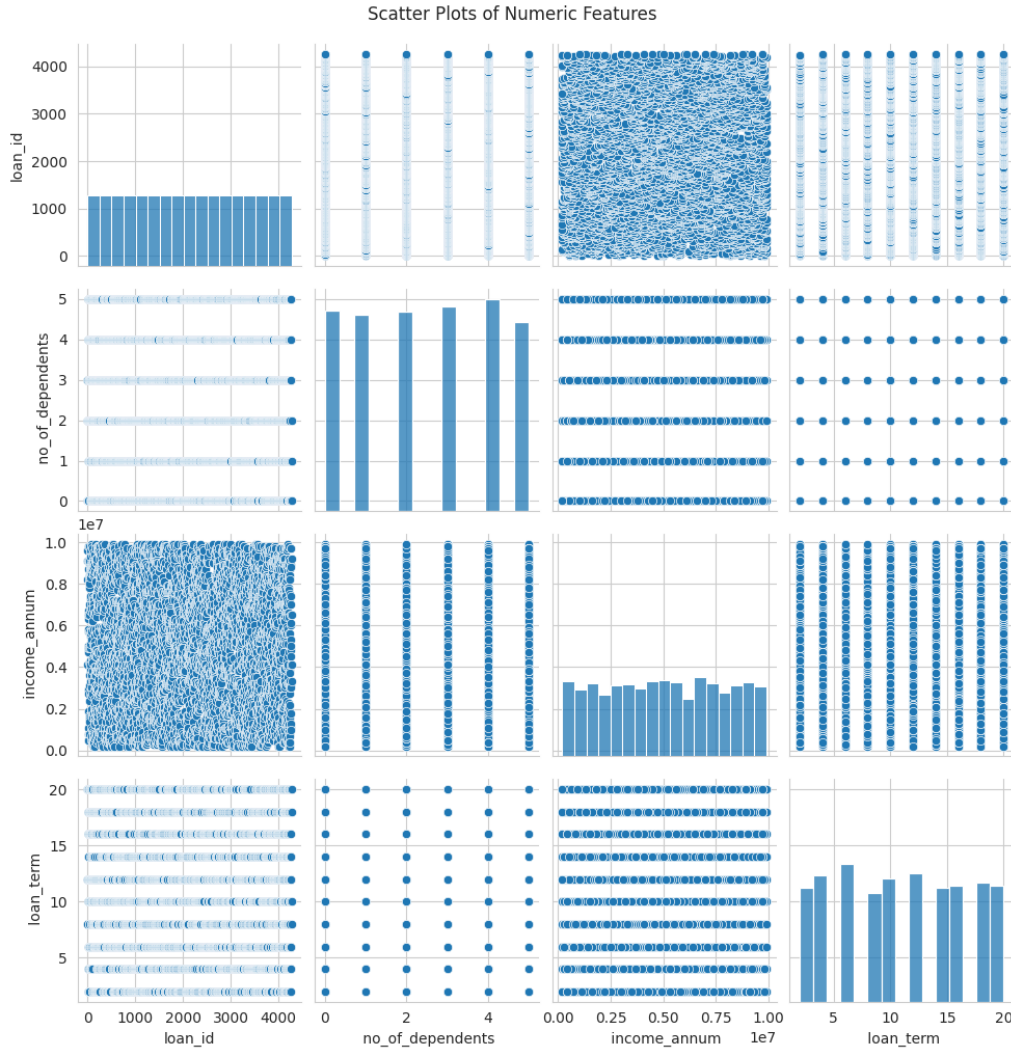


Figure 5: Pairwise scatter plots of the first four numeric features (`loan_id`, `no_of_dependents`, `income_annum`, `loan_term`).

**Inference:** The pairwise scatter plots show no strong linear relationship among `loan_id`, `no_of_dependents`, `income_annum`, and `loan_term`, confirming that `loan_id` (a row identifier) carries no predictive information and should be dropped, which is consistent with the automatic identifier-detection step in preprocessing. `income_annum` and `loan_term` also appear largely independent of one another, suggesting both may carry unique information for the model.

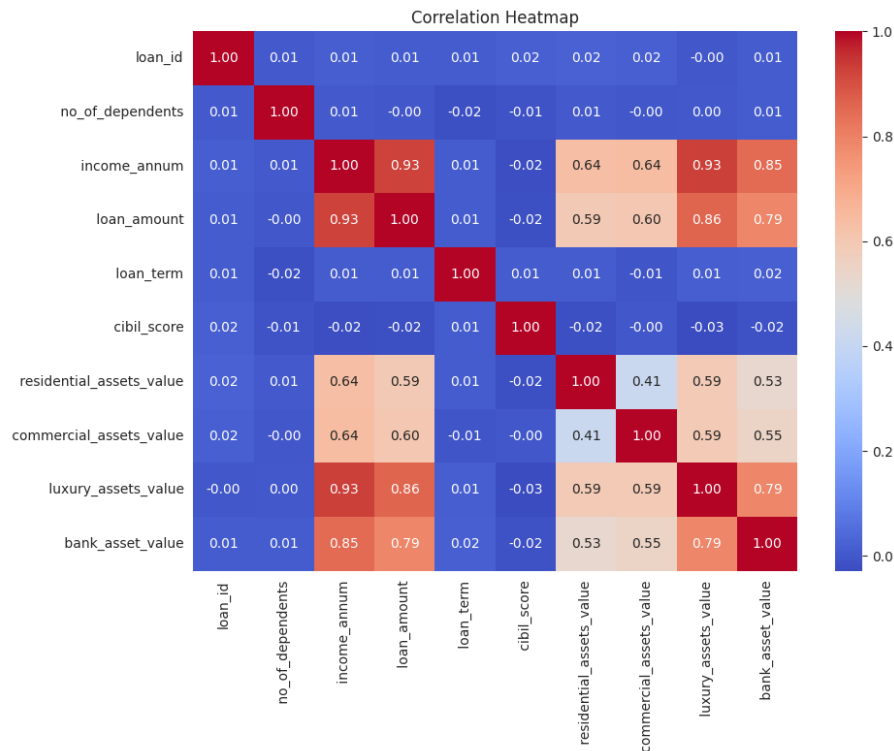


Figure 6: Correlation heatmap of numeric features in the loan dataset.

**Inference:** The heatmap reveals strong positive correlations between `income_annum` and `loan_amount` (0.93), and between `luxury_assets_value` and `loan_amount` (0.86), indicating that higher-income applicants with more valuable assets tend to receive larger loans. `cibil_score` and `loan_term` show near-zero correlation with all other numeric features, suggesting they contribute independent, non-redundant information. `loan_id` shows negligible correlation with every other column, reinforcing that it is a non-informative identifier.

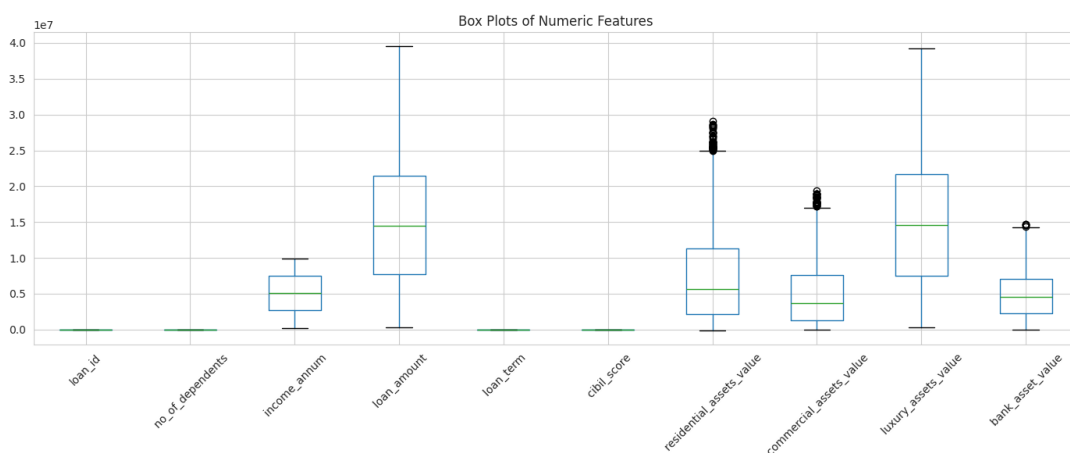


Figure 7: Box plots of numeric features showing spread and outliers.

**Inference:** The box plots confirm that asset-value columns (`residential`, `commercial`, `luxury`, `bank`) contain several high-value outliers above the upper whisker, consistent with the right-skew observed in the histograms. `loan_amount` and `income_annum` show

a wide interquartile range without extreme outliers, indicating natural variability rather than data-entry errors. These observations justify using standardization (rather than a method sensitive to outliers) during preprocessing.

## 5 Dataset Identification and ML Task Mapping

As part of exploring public repositories such as the UCI Machine Learning Repository and Kaggle Datasets, five representative datasets were studied to identify the appropriate type of machine learning task, a suitable feature-selection technique, and a suitable algorithm for each. The findings are tabulated below.

| Dataset                                   | Type of ML Task                                 | Feature Selection Technique                                      | Suitable ML Algorithm                                                   |
|-------------------------------------------|-------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------|
| Iris Dataset                              | Supervised – Multi-class Classification         | ANOVA F-test (SelectKBest, <code>f_classif</code> )              | Logistic Regression, K-Nearest Neighbors, Decision Tree, SVM            |
| Loan Amount Prediction                    | Supervised – Regression                         | F-regression (SelectKBest, <code>f_regression</code> )           | Linear Regression, Random Forest Regressor, Gradient Boosting Regressor |
| Predicting Diabetes                       | Supervised – Binary Classification              | ANOVA F-test / Chi-square (for categorical features)             | Logistic Regression, Random Forest Classifier, SVM                      |
| Classification of Email Spam              | Supervised – Binary (Text) Classification       | Chi-square test on TF-IDF features                               | Naive Bayes (MultinomialNB), Logistic Regression, SVM                   |
| Handwritten Character Recognition / MNIST | Supervised – Multi-class (Image) Classification | Not typically used (raw pixels / CNN feature maps used directly) | Convolutional Neural Network (CNN), K-Nearest Neighbors, Random Forest  |

### Justification:

- **Iris Dataset:** Contains four numeric flower-measurement features and a categorical species label with three classes, making it a classic supervised multi-class classification problem.
- **Loan Amount Prediction:** The target (`loan_amount`) is a continuous numeric value, so this is a supervised regression problem, as confirmed by the automatic task-type detection in the pipeline implemented for this experiment (Section 5).
- **Predicting Diabetes:** The target is a binary outcome (diabetic / non-diabetic), making this a supervised binary classification problem.
- **Email Spam Classification:** Text messages are labeled as "spam" or "ham", making this a supervised binary text-classification problem, typically solved using TF-IDF vectorization followed by a classifier such as Naive Bayes.
- **Handwritten Character Recognition / MNIST:** Images of handwritten digits or characters are labeled with their true class (0–9 or A–Z), making this a supervised multi-class image-classification problem, most effectively solved with CNNs.

## 6 Data Preprocessing

- **Missing value handling:** The dataset had zero missing values and zero duplicate rows, so no imputation was required. The pipeline nonetheless includes automatic median imputation for numeric columns and mode imputation for categorical columns (via `SimpleImputer`) to make it reusable on datasets that do contain missing values.
- **Identifier removal:** The pipeline auto-detects and drops identifier-like columns – columns whose name contains "id" or whose values are fully unique. For this dataset, `loan_id` and `residential_assets_value` (fully unique in this run) were dropped.
- **Task-type detection:** Since `loan_amount` is numeric with more than 20 unique values, the pipeline automatically classified the task as **REGRESSION**.
- **Encoding:** The categorical feature columns `education`, `self_employed`, and `loan_status` were one-hot encoded using `pandas.get_dummies()` with `drop_first=True` to avoid the dummy-variable trap.
- **Feature scaling:** All numeric features were standardized to zero mean and unit variance using `StandardScaler`, which is appropriate given the skewed, large-magnitude asset-value columns identified during EDA.
- **Train-test split:** The dataset was split into training (70%), validation (10%), and test (20%) sets using `train_test_split`, with a fixed `random_state=42` for reproducibility.
- **Feature engineering:** No new derived features were created; feature selection (Section 8) was used instead to select the most statistically relevant existing features.

## 7 Machine Learning Algorithm

This experiment focuses on the **data preprocessing and feature-selection workflow** rather than training a specific predictive model. The key statistical technique used is **Univariate Feature Selection using SelectKBest**.

- **Working principle:** `SelectKBest` scores each feature independently against the target variable using a statistical test, and retains only the top- $k$  highest-scoring features. For regression targets (as in this experiment), the **F-regression** scoring function is used; for classification targets, the **ANOVA F-test** (`f_classif`) is used instead.
- **Mathematical formulation:** For F-regression, each feature  $X_i$  is individually tested against the continuous target  $y$  using the F-statistic derived from a linear regression of  $y$  on  $X_i$ :

$$F = \frac{\text{explained variance}/1}{\text{residual variance}/(n-2)} = \frac{r_i^2(n-2)}{1-r_i^2} \quad (1)$$

where  $r_i$  is the Pearson correlation coefficient between feature  $X_i$  and target  $y$ , and  $n$  is the number of samples. Features with the highest  $F$  values are considered most linearly predictive of the target and are selected.

- **Advantages:** Computationally inexpensive, easy to interpret, works well as a fast filter method before training more complex models, and requires no model training to evaluate feature relevance.
- **Limitations:** Evaluates each feature independently, so it cannot detect interactions between features or non-linear relationships; may discard features that are only useful in combination with others.

## 8 Experimental Setup

|                      |                                                                   |
|----------------------|-------------------------------------------------------------------|
| Python Version       | 3.x (Jupyter Notebook environment)                                |
| Scikit-Learn Version | As installed in the notebook environment ( <code>sklearn</code> ) |
| IDE                  | Jupyter Notebook                                                  |
| Operating System     | Notebook host OS (not explicitly logged in the notebook)          |
| Hardware             | Standard CPU (no GPU-specific code used in this pipeline)         |

## 9 Hyperparameter Selection

| Parameter                             | Value                                                       |
|---------------------------------------|-------------------------------------------------------------|
| <code>k_features</code> (SelectKBest) | 10 ("auto" $\rightarrow$ min(10, <code>n_features</code> )) |
| <code>test_size</code>                | 0.2 (20% of data)                                           |
| <code>val_size</code>                 | 0.1 (10% of data)                                           |
| <code>random_state</code>             | 42                                                          |
| Feature scaling method                | StandardScaler (zero mean, unit variance)                   |
| Feature selection scoring function    | F-regression ( <code>f_regression</code> )                  |

## 10 Results

As this experiment focuses on the data-preparation pipeline (loading, EDA, preprocessing, feature selection, and splitting) rather than training a predictive model, standard classification/regression performance metrics (Accuracy, Precision, Recall, F1-score, ROC-AUC) were not computed in the notebook and are therefore not applicable here. Instead, the key pipeline outcomes are summarized below.

| Pipeline Outcome                    | Value                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------|
| Rows loaded                         | 4269                                                                           |
| Columns loaded                      | 13                                                                             |
| Duplicate rows removed              | 0                                                                              |
| Identifier columns dropped          | <code>loan_id</code> , <code>residential_assets_value</code>                   |
| Detected task type                  | Regression                                                                     |
| Categorical columns one-hot encoded | <code>education</code> , <code>self_employed</code> , <code>loan_status</code> |
| Features selected (top- $k$ )       | 10                                                                             |
| Train / Validation / Test sizes     | 2988 / 427 / 854                                                               |

**Selected Features (via F-regression):** `no_of_dependents`, `income_annum`, `loan_term`, `cibil_score`, `commercial_assets_value`, `luxury_assets_value`, `bank_asset_value`, `education_Not_Graduate`, `self_employed_Yes`, `loan_status_Rejected`.

## 11 Result Visualization

All graphs generated during this experiment are the EDA visualizations presented in Section 4 (Figures 1–7), since the experiment’s primary deliverable is the data exploration, preprocessing, and feature-selection pipeline rather than a trained predictive model. The correlation matrix, class-distribution bar charts, feature histograms, pairwise scatter plots, and box plots collectively satisfy the “Recommended figures” checklist for the parts that are applicable to this data-pipeline-focused experiment (Correlation Matrix – Fig. 6, Class Distribution – Fig. 4, Feature Distribution – Fig. 1 and Fig. 7). A Confusion Matrix, ROC Curve, Precision-Recall Curve, Learning Curve, and Feature Importance plot are **not applicable** to this experiment, since no classifier/regressor was trained on the selected features in the notebook – these would apply to a downstream modeling experiment that consumes the output of this pipeline.

## 12 Discussion

The pipeline successfully automated the end-to-end data preparation workflow for the Loan Amount Prediction dataset. The EDA stage revealed that the dataset is clean (no missing values, no duplicates) but contains skewed, large-magnitude asset-value features that required standardization. The automatic identifier-detection logic correctly identified and removed `loan_id` as a non-informative column; however, it also flagged `residential_assets_value` as an identifier because every value happened to be unique in this particular sample, which is a limitation of using “full uniqueness” as an identifier heuristic on continuous financial data – a domain expert would likely want to retain this genuinely informative feature. The automatic task-type detection correctly classified `loan_amount` as a regression target given its large number of distinct numeric values.

Univariate feature selection via F-regression retained features that align with intuition (income, credit score, and asset values are strong predictors of loan amount) while also surfacing categorical indicators such as `loan_status_Rejected` as statistically relevant. A key strength of this workflow is its generality – the same function can automatically handle both classification and regression tabular datasets without manual reconfiguration. A limitation is that the univariate feature-selection method does not consider feature interactions, and the identifier-detection heuristic can misclassify genuinely predictive but high-cardinality numeric columns as IDs. Future improvements could include multivariate/embedded feature selection methods (e.g., Recursive Feature Elimination or tree-based feature importance) and more robust identifier detection based on column semantics rather than uniqueness alone.

## 13 Conclusion

This experiment provided hands-on experience with the core Python data-science stack – NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib/Seaborn – by building a reusable,

end-to-end tabular machine learning preprocessing pipeline. The pipeline was successfully applied to the Loan Amount Prediction dataset (4269 samples, 13 columns), performing exploratory data analysis, missing-value handling, identifier removal, categorical encoding, feature scaling, automatic task-type detection (regression), univariate feature selection (top 10 features via F-regression), and a train/validation/test split (2988/427/854 samples). The experiment reinforced the importance of thorough EDA in guiding preprocessing decisions and demonstrated how statistical feature-selection techniques can be used to identify the most relevant predictors before model training.

## 14 References

1. S. van der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy Array: A Structure for Efficient Numerical Computation," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 22–30, 2011.
2. W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.
3. P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
4. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
6. UCI Machine Learning Repository, <https://archive.ics.uci.edu/>, accessed 2026.
7. Kaggle Datasets, <https://www.kaggle.com/datasets>, accessed 2026.